Nithin Gowtham Saravanan

FPGA HARDWARE BRING-UP USING VIVADO

Table of Contents

Introduction 2
Procedure 2
1. Project Setup and Environment Initialization 2
2. Switch-to-LED Design 2
3. 4-Bit Counter Design 3
4. Jackpot Game Design 3
Results 4
Conclusion 4
Knowledge Sharing 5
Appendix 6
APPENDIX A (switch.v) 6
APPENDIX B (counter.v) 6
APPENDIX C (jackpot.v) 7
APPENDIX D (switch.xdc) 8
APPENDIX E (counter.xdc) 8
APPENDIX F (jackpot.xdc) 9

Introduction

The first lab session is designed to introduce the FPGA design workflow using Xilinx Vivado and the ZYBO Z7-10 development board. The primary objective of this lab was to gain familiarity with hardware design using Verilog, applying Xilinx Design Constraints (XDC) for pin assignments, generating a bitstream, and programming the FPGA device.

The lab consisted of three major components. First, a simple combinational design was implemented to map on-board DIP switches to LEDs. Next, a 4-bit counter controlled by push buttons was developed using a divided system clock. Finally, a jackpot game was designed in which LEDs glow sequentially in a one-hot pattern and a winning condition is detected based on user interaction. These exercises reinforced fundamental concepts of synchronous digital design and FPGA-based system development.

Procedure

1. Project Setup and Environment Initialization

1. A dedicated directory named lab1 was created in the home workspace to organize all files related to Laboratory Exercise #1.
2. The Vivado 2023.1 environment was initialized by sourcing the required setup script from the terminal.
3. The Vivado Design Suite was launched from the terminal.
4. A new RTL project was created in Vivado with Verilog selected as the target language.
5. The FPGA device xc7z010clg400-1, corresponding to the ZYBO Z7-10 board, was selected from the Parts list.

2. Switch-to-LED Design

1. A Verilog source file named **switch.v** was created and defined with a 4-bit input port for the DIP switches and a 4-bit output port for the LEDs.
2. A continuous assignment was used to directly map each DIP switch to its corresponding LED (Appendix A).
3. A constraint file named **switch.xdc** was created to assign FPGA pins for the DIP switches and LEDs on the ZYBO Z7-10 board (Appendix D).
4. The constraint file was added to the project through the *Add Sources* option under the Constraints section.

5. The **switch.v** module was verified as the top module in the Sources window.
6. Synthesis and implementation were run to generate the bitstream.
7. The FPGA was programmed using the Hardware Manager.
8. Correct operation was verified by toggling the DIP switches and observing the LEDs, and the design was demonstrated to the TA.

3. 4-Bit Counter Design

1. A new Verilog source file named **counter.v** was created and configured as the top module in the Sources window.
2. A 4-bit up/down counter was implemented using the on-board LEDs for output and push buttons for control (Appendix B).
3. A clock divider was implemented to generate an approximately one-second enable pulse from the 125 MHz on-board clock.
4. A constraint file named **counter.xdc** was created to map the push buttons, LEDs, clock, and reset signals to the appropriate FPGA pins (Appendix E).
5. The constraint file was added to the project, and the bitstream was regenerated.
6. The FPGA was programmed, and correct counter operation was verified by observing LED behavior in response to button inputs.
7. The working counter design was demonstrated to the TA.

4. Jackpot Game Design

1. A new Verilog source file named **jackpot.v** was created and set as the top module in the Sources window.
2. A one-hot LED rotation scheme was implemented such that only one LED is active at a time (Appendix C).
3. Edge detection logic was added by storing previous DIP switch values and comparing them with current values to detect rising transitions.
4. A state variable was implemented to latch the jackpot condition once the correct switch was toggled at the correct time.
5. A constraint file named **jackpot.xdc** was created to assign FPGA pins for the DIP switches, LEDs, clock, and reset signals (Appendix F).
6. The constraint file was added to the project, and synthesis and implementation were run to generate the bitstream.
7. The FPGA was programmed, and correct jackpot operation was verified.
8. The working jackpot design was demonstrated to the TA.

Results

A series of FPGA-based hardware designs were developed and implemented on the ZYBO Z7-10 board using the Vivado design flow. In the initial task, a combinational hardware module was created to map on-board DIP switches directly to LEDs, verifying correct pin assignment and basic RTL functionality. The design was synthesized, implemented, and programmed successfully, and correct operation was confirmed by toggling the DIP switches and observing the corresponding LED behavior.

Subsequently, a 4-bit up/down counter was implemented using push buttons for control and LEDs for output. A clock divider was designed to generate approximately one-second update intervals from the 125 MHz system clock, enabling human-observable LED transitions. The counter incremented, decremented, or held its value based on button inputs, and correct functionality was verified on hardware.

Finally, a jackpot game was implemented in which LEDs illuminated sequentially in a one-hot pattern. Edge detection logic was used to detect rising transitions of the DIP switches, and a state variable was used to latch the jackpot condition. When a switch corresponding to the active LED was toggled at the correct time, the jackpot condition was detected and all LEDs illuminated. The design operated as expected and was demonstrated to the TA for verification.

(Source code and constraint files for all designs are provided in the Appendix.)

Conclusion

In summary, this laboratory exercise provided practical experience with FPGA design and implementation using Xilinx Vivado and the ZYBO Z7-10 development platform. Through the development of progressively complex hardware modules, key digital design concepts such as combinational logic, clock division, edge detection, and state-based control were reinforced.

The lab emphasized the importance of synchronous design practices, proper use of constraint files, and systematic verification on physical hardware. The knowledge and skills gained from this exercise establish a strong foundation for more advanced FPGA-based and embedded system design tasks in subsequent laboratory sessions.

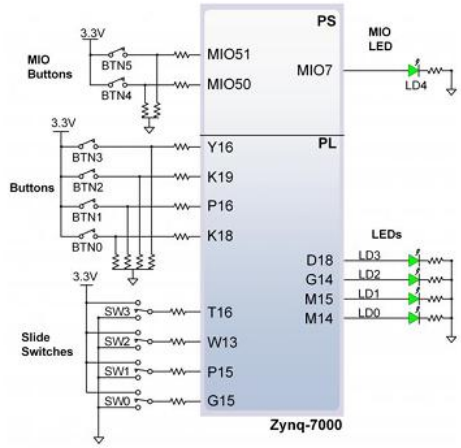
Nithin Gowtham Saravanan

Knowledge Sharing

a) How are the user push-buttons wired on the ZYBO Z7-10 board (i.e., what pins on the FPGA do each of them correspond to, and are the signals pulled up or down)?

The ZYBO Z7-10 board includes four user push-buttons connected to the Zynq Processing Logic (PL) through series resistors, as illustrated in reference manual. These series resistors protect the FPGA I/O pins from damage in the event of accidental short circuits, such as when a pin is mistakenly configured as an output.

The user push-buttons BTN0–BTN3 are wired as momentary switches that are pulled down by default. When the buttons are not pressed, they output a logic low (0), and when pressed, they generate a logic high (1) by connecting the signal to 3.3 V.



Based on the figure and constraint assignments used in this lab, the push-buttons correspond to the following FPGA pins on the Zynq PL:

- BTN0 → K18
- BTN1 → P16
- BTN2 → K19
- BTN3 → Y16

b) What is the purpose of an edge detection circuit, and how should it have been used in this lab?

An edge detection circuit is used to identify a transition in a digital signal, such as a change from logic low to logic high, rather than responding to the steady-state value of the signal. This allows the system to recognize a single event at the moment it occurs and prevents repeated or unintended activations when the signal remains asserted across multiple clock cycles.

In this laboratory, edge detection was required in the jackpot game to detect the rising transition of the DIP switches. By comparing the current switch values with their values from the previous evaluation cycle, the design could determine when a switch was toggled from OFF to ON. This ensured that the jackpot condition was triggered only once and only when the correct switch was flipped at the exact moment its corresponding LED was active.

The clock divider used in the counter and jackpot designs does not perform edge detection. Instead, it operates synchronously on the rising edge of the system clock to increment a counter. When the counter reaches a value corresponding to one second, a timing enable pulse is generated. This enable signal is then used to periodically sample push-button inputs in the counter design and DIP switch states in the jackpot design at a human-visible rate.

Appendix

APPENDIX A (switch.v)

```
module switch(
    input [3:0] SWITCHES,
    output [3:0] LEDS
    );
    // Turn on the LEDs when the DIP switch is HIGH
    assign LEDS[3:0] = SWITCHES[3:0];
endmodule
```

APPENDIX B (counter.v)

```
module counter(
    input [1:0] BUTTONS,
    output reg [3:0] LEDS,
    input clock,
    input reset
    );

    reg one_sec_pulse;
    reg [27:0] clk_pulse;

    // Use clock divider to control the LEDs blinking
    always @(posedge clock) begin
        if (clk_pulse > 125000000) begin    // Default 125MHz = 8ns; => 1s/8ns = 125000000 cycles
            one_sec_pulse <= 1'b1;
            clk_pulse <= 0;
        end
        else begin
            clk_pulse <= clk_pulse + 1;
            one_sec_pulse <= 1'b0;
        end
    end


    always @(posedge clock) begin
        if (reset)                  // When reset button is pressed, all LEDs should be turned OFF
            LEDS <= 4'b0000;
        else if (one_sec_pulse) begin
            if ( BUTTONS[0] && !BUTTONS[1])
                LEDS <= LEDS + 1;   // increment the counter
            else if ( !BUTTONS[0] && BUTTONS[1])
                LEDS <= LEDS - 1;   // decrement the counter
            else
                LEDS <= LEDS;       // stay in the same state
        end
    end
endmodule
```

APPENDIX C (jackpot.v)

/home/grads/n/nithingowtham25/Spring_2026/ECEN749/lab_1/lab_1.srcs/sources_1/new/jackpot.v

```
module jackpot(
    input [3:0] DIP,
    input reset,
    input clock,
    output [3:0] LEDS
    );

    reg one_sec_pulse;
    reg [27:0] clk_pulse;

    // Use clock divider to control the LEDs blinking
    always @(posedge clock) begin
        if (clk_pulse > 125000000) begin    // Default 125MHz = 8ns; => 1s/8ns = 125000000 cycles
            one_sec_pulse <= 1'b1;
            clk_pulse <= 0;
        end
        else begin
            clk_pulse <= clk_pulse + 1;
            one_sec_pulse <= 1'b0;
        end
    end

    reg jackpot = 1'b0;                     // game status
    reg [3:0] prev_switch_val = 4'b0000;    // previous cycle switch status
    reg [3:0] led_status = 4'b0001;         // led status

    always @(posedge one_sec_pulse) begin
        // When the reset button is pressed, initialize the switch and led status
        if(reset) begin
            jackpot <= 1'b0;
            prev_switch_val <= 4'b0000;
            led_status <= 4'b0001;
        end
        else if (one_sec_pulse) begin
            // If the previous switch = OFF and current switch = ON and corresponding LED is glowing => JACKPOT
            if( jackpot != 1'b1) begin
                if (  ( !prev_switch_val[0] && DIP[0] && led_status[0] ) ||
                      ( !prev_switch_val[1] && DIP[1] && led_status[1] ) ||
                      ( !prev_switch_val[2] && DIP[2] && led_status[2] ) ||
                      ( !prev_switch_val[3] && DIP[3] && led_status[3] )
                    )  begin
                    jackpot <= 1'b1;
                    led_status <= 4'b1111;
                end
                else begin
                    if( led_status == 4'b1000) begin        //for circular shift
                            led_status <= 4'b0001;
                    end
                    else begin
                        led_status <= (led_status << 1'b1); // sequential blinking of LEDs
                    end
                end
            end
        end

        prev_switch_val[3:0] <= DIP[3:0];    // assign previous_switch_values as current switch value
    end

    assign LEDS[3:0] = led_status[3:0];     // final LED status

endmodule
```

APPENDIX D (switch.xdc)

/home/grads/n/nithingowtham25/Spring_2026/ECEN749/lab_1/switch.xdc

```
##Switches
set_property PACKAGE_PIN G15 [ get_ports {SWITCHES[0] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {SWITCHES[0] } ]

set_property PACKAGE_PIN P15 [ get_ports {SWITCHES[1] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {SWITCHES[1] } ]

set_property PACKAGE_PIN W13 [ get_ports {SWITCHES[2] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {SWITCHES[2] } ]

set_property PACKAGE_PIN T16 [ get_ports {SWITCHES[3] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {SWITCHES[3] } ]


##LEDs
set_property PACKAGE_PIN M14 [ get_ports {LEDS[0] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[0] } ]

set_property PACKAGE_PIN M15 [ get_ports {LEDS[1] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[1] } ]

set_property PACKAGE_PIN G14 [ get_ports {LEDS[2] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[2] } ]

set_property PACKAGE_PIN D18 [ get_ports {LEDS[3] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[3] } ]
```

APPENDIX E (counter.xdc)

/home/grads/n/nithingowtham25/Spring_2026/ECEN749/lab_1/counter.xdc

```
##BUTTONs
set_property PACKAGE_PIN K18 [ get_ports {BUTTONS[0] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {BUTTONS[0] } ]

set_property PACKAGE_PIN P16 [ get_ports {BUTTONS[1] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {BUTTONS[1] } ]


##LEDs
set_property PACKAGE_PIN M14 [ get_ports {LEDS[0] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[0] } ]

set_property PACKAGE_PIN M15 [ get_ports {LEDS[1] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[1] } ]

set_property PACKAGE_PIN G14 [ get_ports {LEDS[2] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[2] } ]

set_property PACKAGE_PIN D18 [ get_ports {LEDS[3] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[3] } ]


##CLOCK
set_property PACKAGE_PIN K17 [ get_ports clock ]
set_property IOSTANDARD LVCMOS33 [ get_ports clock ]


##RESET
set_property PACKAGE_PIN Y16 [ get_ports reset ]
set_property IOSTANDARD LVCMOS33 [ get_ports reset ]
```

APPENDIX F (jackpot.xdc)

/home/grads/n/nithingowtham25/Spring_2026/ECEN749/lab_1/jackpot.xdc

```
##DIP switches
set_property PACKAGE_PIN G15 [ get_ports {DIP[0] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {DIP[0] } ]

set_property PACKAGE_PIN P15 [ get_ports {DIP[1] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {DIP[1] } ]

set_property PACKAGE_PIN W13 [ get_ports {DIP[2] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {DIP[2] } ]

set_property PACKAGE_PIN T16 [ get_ports {DIP[3] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {DIP[3] } ]


##LEDs
set_property PACKAGE_PIN M14 [ get_ports {LEDS[0] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[0] } ]

set_property PACKAGE_PIN M15 [ get_ports {LEDS[1] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[1] } ]

set_property PACKAGE_PIN G14 [ get_ports {LEDS[2] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[2] } ]

set_property PACKAGE_PIN D18 [ get_ports {LEDS[3] } ]
set_property IOSTANDARD LVCMOS33 [ get_ports {LEDS[3] } ]


##CLOCK
set_property PACKAGE_PIN K17 [ get_ports clock ]
set_property IOSTANDARD LVCMOS33 [ get_ports clock ]


##RESET
set_property PACKAGE_PIN Y16 [ get_ports reset ]
set_property IOSTANDARD LVCMOS33 [ get_ports reset ]
```